1. Introduction

Après avoir établi la vision, le périmètre MVP et l'architecture technique du backend de Quantara, cette étape cruciale se concentre sur la définition de la **carte globale des routes API**. L'objectif est de créer une interface claire et prévisible entre le frontend et le backend, garantissant une communication fluide et une implémentation structurée.

Ce document ne vise pas à détailler chaque endpoint de manière exhaustive, mais plutôt à répondre aux questions fondamentales :

- Quelles sont les grandes familles de routes nécessaires ?
- Comment ces routes sont-elles organisées et versionnées ?
- Quels endpoints sont indispensables pour le MVP ?
- Comment ces routes s'alignent-elles avec la structure du frontend ?

Une spécification claire du routing API est essentielle pour la collaboration entre les équipes frontend et backend, la génération de documentation automatique et la maintenabilité du système à long terme.

2. Objectif du Routing API Global

Le routing API global de Quantara doit permettre de construire une API :

- **Lisible et Prévisible :** Facile à comprendre et à utiliser pour les développeurs frontend.
- **Versionnée :** Capable d'évoluer sans casser les clients existants.

- **Modulaire :** Reflétant la structure des modules métier du backend.
- **Cohérente :** Appliquant des conventions de nommage et des patterns uniformes.

L'objectif est d'éviter les écueils courants tels qu'une API désordonnée, des endpoints incohérents, des noms de routes ambigus ou des patterns de communication variables entre les modules. Une API bien conçue donne immédiatement une impression de clarté et de professionnalisme.

3. Base du Routing API

Toutes les routes du backend de Quantara seront regroupées sous une base commune et versionnée :

```
/api/v1/
```

Justification :

- **Versionnement Précoce :** Intégrer le versionnement dès le début (`v1/`) anticipe les évolutions futures de l'API, permettant d'introduire de nouvelles versions (`v2/` , `v3/`) sans impacter les clients utilisant la version actuelle.
- **Stabilité Frontend :** Cette approche évite de casser le frontend lors de modifications majeures du backend.
- **Professionnalisme :** Adopter une structure versionnée est une bonne pratique standard dans le développement d'API professionnelles.

Par conséquent, tous les endpoints du MVP de Quantara commenceront par `/api/v1/...` .

4. Grandes Familles de Routes

Le backend MVP de Quantara est structuré autour des grandes familles de routes suivantes, qui correspondent aux domaines fonctionnels clés de l'application :

A. Authentification (`auth/`)

Routes dédiées à la gestion de l'accès utilisateur :

- Connexion (`login`)
- Récupération de l'utilisateur courant (`me`)
- Déconnexion (`logout`, à implémenter si nécessaire pour l'invalidation de session)
- Enregistrement (`register`, à reporter après le MVP)

B. Données de Marché (`assets/`, `charting/`)

Routes pour l'accès aux informations sur les actifs financiers et leurs données historiques :

- Liste des actifs disponibles
- Détail d'un actif spécifique
- Données de bougies (candles) pour les graphiques

C. Simulation de Trading (`simulation/`, `portfolio/`, `trades/`)

Routes pour l'exécution de trades simulés et la consultation du portefeuille virtuel :

- Exécution d'un trade simulé
- Résumé du portefeuille virtuel
- Positions ouvertes
- Historique des trades

D. Éducation (`courses/`, `lessons/`)

Routes pour la gestion et la consultation du contenu pédagogique :

- Liste des cours
- Détail d'un cours
- Contenu d'une leçon

E. Progression Utilisateur (`progress/`)

Routes pour le suivi de l'avancement de l'utilisateur dans les cours et les simulations :

- Progression globale de l'utilisateur
- Statistiques simples de progression

F. Profil et Paramètres Utilisateur (`profile/`, `settings/`)

Routes pour la gestion des informations personnelles et des préférences de l'utilisateur :

- Informations du profil utilisateur
- Préférences et paramètres de l'application

5. Routing Global Recommandé pour le MVP

La structure globale recommandée pour le routing API du MVP de Quantara est la suivante. Elle reflète une organisation logique et modulaire, alignée avec les besoins du frontend.

```
/api/v1/
├── auth/
│   ├── POST /login              # Connexion de l'utilisateur
│   └── GET /me                  # Récupération des informations de
l'utilisateur courant
│
├── dashboard/
│   └── GET /summary             # Résumé agrégé pour le tableau de bord
│
├── assets/
│   ├── GET /                    # Liste de tous les actifs
│   ├── GET /{symbol}            # Détail d'un actif spécifique
│   └── GET /{symbol}/candles    # Données de bougies pour un actif
│
├── simulation/
│   └── POST /trades             # Exécution d'un trade simulé
│
├── portfolio/
│   ├── GET /                    # Résumé du portefeuille utilisateur
│   ├── GET /positions           # Liste des positions ouvertes
│   └── GET /performance         # Données de performance du portefeuille
│
├── trades/
│   └── GET /                    # Historique des trades de l'utilisateur
│
├── courses/
│   ├── GET /                    # Liste de tous les cours
│   └── GET /{course_id}         # Détail d'un cours spécifique
│
├── lessons/
│   ├── GET /{lesson_id}         # Contenu d'une leçon spécifique
│   └── POST /{lesson_id}/complete # Marquer une leçon comme complétée
│
├── progress/
│   └── GET /                    # Progression globale de l'utilisateur
│
├── profile/
│   └── GET /                    # Informations du profil utilisateur
│
└── settings/
    ├── GET /                    # Préférences utilisateur
    └── PATCH /                  # Mise à jour des préférences utilisateur
```

Note sur `/api/v1/assets/{symbol}/candles` vs `/api/v1/charting/{symbol}/candles` : Bien que la séparation via `/charting/` soit structurellement plus propre pour isoler la logique de graphique, pour un MVP, l'intégration des données de bougies directement sous `/assets/{symbol}/candles` peut simplifier l'implémentation initiale. La décision finale dépendra de la complexité anticipée de la logique de `charting` au-delà du MVP.

Endpoint `dashboard/summary` : La création d'un endpoint agrégé `/api/v1/dashboard/summary` est fortement recommandée pour le MVP. Cette approche simplifie considérablement le développement frontend en réduisant le nombre d'appels API nécessaires pour afficher le tableau de bord principal, tout en permettant au backend d'optimiser la récupération et l'agrégation des données.

6. Fonctionnalités Reportées après le MVP

Afin de maintenir la focalisation sur le MVP et d'éviter la surcharge, plusieurs endpoints et fonctionnalités seront volontairement reportés :

- `POST /auth/register` : L'enregistrement utilisateur peut être géré manuellement ou via un processus simplifié au début.
- Flux complets de `forgot/reset password`.
- Gestion de `refresh tokens` pour l'authentification.
- Endpoints avancés pour le `charting` (indicateurs techniques complexes, comparaisons).
- Détails de trade complets et fonctionnalités de recherche avancée.
- Endpoints pour les `insights` (recommandations, analyses comportementales).
- Fonctionnalités `analytics` avancées.
- Endpoints d'administration.

7. Règles de Nommage API

Pour garantir la cohérence et la clarté de l'API, les règles de nommage suivantes seront appliquées :

- **Utiliser des noms clairs et descriptifs.**
- **Utiliser des pluriels pour les collections de ressources** (ex: `/assets`, `/trades`, `/courses`).
- **Utiliser des verbes uniquement lorsque l'action est prédominante** et ne peut être représentée par une ressource (ex: `POST /auth/login`, `POST /lessons/{lesson_id}/complete`).

Exemples de bonnes pratiques :

- `/api/v1/assets` (liste des actifs)
- `/api/v1/trades` (historique des trades)
- `/api/v1/courses` (liste des cours)
- `/api/v1/progress` (progression utilisateur)
- `/api/v1/profile` (profil utilisateur)

Exemples à éviter :

- `/api/v1/getAssets` (verbe inutile)
- `/api/v1/doTrade` (verbe inutile)
- `/api/v1/fetchUserProgress` (verbe inutile)

L'approche privilégiée est une API RESTful, où les ressources sont les noms et les méthodes HTTP (GET, POST, PUT, PATCH, DELETE) définissent l'action.

8. Conclusion

Cette étape a permis de définir une carte claire et structurée du routing API global pour le backend de Quantara. En spécifiant les grandes familles de routes, les endpoints

essentiels pour le MVP, les fonctionnalités reportées et les conventions de nommage, nous avons établi un contrat d'interface solide entre le frontend et le backend.

Cette spécification est une étape essentielle avant de passer à la définition détaillée des modèles de données et des schémas Pydantic, puis à l'implémentation concrète des endpoints. Elle garantit que le développement de l'API sera cohérent, maintenable et aligné avec les objectifs du produit.